

00 Что такое SPA и почему не обычные HTML-файлы?

В классическом сайте при каждом переходе (например, с `index.html` на `about.html`) браузер полностью стирает экран, заново делает сетевой запрос и заново скачивает шапку, подвал и стили.

SPA (Single Page Application) работает иначе:

- На сервере лежит **всего один** HTML-файл (обычно пустой `<div id="root"></div>`).
 - Всю отрисовку кнопок, страниц и карточек берет на себя JavaScript.
 - Когда пользователь кликает на «О нас», JS просто удаляет с экрана блок главной страницы и рисует блок «О нас».
- Страница не перезагружается, сайт работает со скоростью приложения.**

01 Подготовка: Установка Node.js

Чтобы собирать современный фронтенд, нам нужен инструмент выполнения JavaScript вне браузера — **Node.js** (он включает пакетный менеджер **npm**).

1. Зайдите на официальный сайт nodejs.org.
2. Скачайте и установите версию **LTS** (Long Term Support).
3. Откройте терминал (VS Code Terminal, Git Bash или командную строку) и проверьте установку:

```
$ node -v
v20.x.x (или новее)

$ npm -v
10.x.x
```

02 Создание первого проекта через Vite

Раньше использовали устаревший `create-react-app`. Сегодня стандартом индустрии является **Vite** — он запускает локальный сервер за миллисекунды.

Выполните в терминале команду для создания проекта:

```
$ npm create vite@latest my-react-app -- --template react
```

Перейдите в созданную папку, установите зависимости и запустите локальный сервер:

```
$ cd my-react-app
$ npm install
$ npm run dev
```

🔔 Терминал покажет адрес вроде `http://localhost:5173/`. Откройте его в браузере — ваш первый React-проект уже запущен!

03 Разбор файлов: Что лежит внутри проекта?

Откройте созданную папку в редакторе (например, VS Code). Вот что там находится:

```
my-react-app/
├─ index.html      // Единственный HTML-файл проекта
├─ package.json    // Список библиотек и скриптов запуска
├─ vite.config.js  // Конфигурация сборщика
└─ src/           // Папка с нашим исходным кодом
   ├─ main.jsx     // Точка входа: монтирует React в HTML
   ├─ App.jsx      // Главный компонент страницы
   └─ App.css      // Стили для компонента
```

Как React попадает в HTML?

В файле `index.html` есть всего одна строчка в `<body>`:

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

А в файле `src/main.jsx` React находит этот блок и говорит: «Теперь я управляю всем, что внутри!»:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

04 Что такое Компонент и правила JSX

Компонент — это обычная функция JavaScript, которая называется с **Заглавной буквы** и возвращает кусок HTML-разметки (называемый **JSX**).

Очистите файл `src/App.jsx` и напишите простой компонент:

```
// src/App.jsx
function App() {
  const title = "Мой первый сайт на React";

  return (
    <div className="container">
      <h1>{title}</h1>
      <p>Здесь мы пишем обычную верстку, но с суперсилой JS!</p>
    </div>
  );
}

export default App;
```

3 Главных правила синтаксиса JSX:

- Вместо `class` пишем `className` (потому что слово `class` занято в JS).
- Любые переменные и код JS внутри верстки оборачиваются в фигурные скобки `{title}`.
- Все одиночные теги обязаны закрываться слешем: ``, `<input />`, `<br />`.

05 Props: Как передавать данные в компоненты

Чтобы не дублировать верстку карточек или кнопок, мы создаем переиспользуемый компонент и передаем в него параметры — **Props** (свойства).

Создадим компонент карточки товара `ProductCard`:

```
// Компонент-карточка принимает props (деструктурируем имя и цену)
function ProductCard({ title, price, isStock }) {
  return (
    <div className="card" style={{ border: '1px solid #ccc', padding: '10px', margin: '10px 0' }}>
      <h3>{title}</h3>
      <p>Цена: {price} руб.</p>
      <span>{isStock ? "✅ В наличии" : "❌ Под заказ"}</span>
    </div>
  );
}

// Главный компонент использует ProductCard несколько раз с разными данными
function App() {
  return (
    <div>
      <h1>Каталог товаров</h1>
      <ProductCard title="Смартфон" price={45000} isStock={true} />
      <ProductCard title="Ноутбук" price={85000} isStock={false} />
      <ProductCard title="Наушники" price={5000} isStock={true} />
    </div>
  );
}

export default App;
```

06 State (useState): Реактивность и динамика

Если создать обычную переменную `let count = 0` и увеличивать её в клике, браузер **не обновит экран**. React просто не знает, что значение изменилось.

Для динамических данных используется специальный хук `useState`:

```
import { useState } from 'react';

function Counter() {
  // 1. count — сама переменная состояния
  // 2. setCount — функция, которая меняет значение и заставляет React перерисовать экран
  const [count, setCount] = useState(0);

  function handleIncrement() {
    setCount(count + 1); // React видит вызов и обновляет DOM!
  }

  function handleReset() {
    setCount(0);
  }

  return (
    <div style={{ padding: '20px', background: '#f1f5f9', borderRadius: '8px' }}>
      <h2>Текущий счет: {count}</h2>
      <button onClick={handleIncrement}>+ Увеличить</button>
      <button onClick={handleReset} style={{ marginLeft: '10px' }}>Сбросить</button>
    </div>
  );
}

export default Counter;
```

07 Формы и Управляемые Инпуты (Controlled Inputs)

В чистом JS мы искали инпут через `document.querySelector('input').value`. В React мы связываем значение инпута с состоянием (State):

```
import { useState } from 'react';

function SimpleForm() {
  const [name, setName] = useState('');
  const [submittedText, setSubmittedText] = useState('');

  function handleSubmit(e) {
    e.preventDefault(); // Останавливаем перезагрузку страницы браузером
    setSubmittedText(`Привет, ${name}! Форма успешно отправлена.`);
    setName(''); // Очищаем поле
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Ваше имя:</label>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Введите имя..."
      />
      <button type="submit">Отправить</button>

      {submittedText && <p style={{ color: 'green', marginTop: '10px' }}>{submittedText}</p>}
    </form>
  );
}

export default SimpleForm;
```

08 Списки и массивы: Метод .map() и ключ key

Чтобы вывести массив элементов (список новостей, товаров или задач), используется нативный метод JavaScript

`.map()`.

```
function TodoList() {
  const tasks = [
    { id: 1, text: "Установить Node.js и Vite" },
    { id: 2, text: "Понять пропсы и состояние" },
    { id: 3, text: "Собрать свое первое SPA" }
  ];

  return (
    <ul>
      {tasks.map((task) => (
        // Атрибут key ОБЯЗАТЕЛЕН для каждого элемента списка!
        <li key={task.id}>
          ✨ {task.text}
        </li>
      ))}
    </ul>
  );
}

export default TodoList;
```

Зачем нужен 'key'? Он помогает React понимать, какой конкретно пункт списка был добавлен, удален или перемещен, чтобы не перерисовывать весь список целиком.

09 Собираем мини-SPA: Переключение экранов без перезагрузки

Давайте объединим полученные знания и соберем простое SPA с переключением табов (Главная / Товары / Контакты) в рамках одного файла:

```
import { useState } from 'react';

function MiniSpaApp() {
  const [currentTab, setCurrentTab] = useState('home');

  return (
    <div className="app-wrap">
      {/* Навигационное меню */}
      <nav style={{ display: 'flex', gap: '10px', marginBottom: '20px' }}>
        <button onClick={() => setCurrentTab('home')}>Главная</button>
        <button onClick={() => setCurrentTab('products')}>Каталог</button>
        <button onClick={() => setCurrentTab('contact')}>Контакты</button>
      </nav>

      {/* Условный рендеринг нужного экрана */}
      <main>
        {currentTab === 'home' && (
          <section>
            <h2>Добро пожаловать на Главную страницу</h2>
            <p>Это SPA работает без единой перезагрузки окна!</p>
          </section>
        )}
      </main>
    </div>
  );
}
```

```
        {currentTab === 'products' && (
          <section>
            <h2>Наши продукты</h2>
            <p>Здесь отображается список товаров из массива данных.</p>
          </section>
        )}

        {currentTab === 'contact' && (
          <section>
            <h2>Свяжитесь с нами</h2>
            <p>Телефон: +7 (999) 000-00-00</p>
          </section>
        )}
      </main>
    </div>
  );
}

export default MiniSpaApp;
```

10 Сборка в продакшен (Build) и деплой

Когда сайт готов к публикации на реальный хостинг (например, Vercel, Netlify или обычный сервер):

1. Остановите локальный сервер нажатием `Ctrl + C` в терминале.
2. Выполните команду сборки:

```
$ npm run build
```

Vite превратит все файлы `.jsx` в оптимизированный минифицированный HTML, CSS и JS в папке `dist/`.

Публикация на Vercel за 1 минуту:

- Зарегистрируйтесь на [Vercel.com](https://vercel.com).
- Или установите утилиту Vercel в терминале: `npm i -g vercel` и введите `vercel` в папке проекта.
- Ваш сайт мгновенно получит бесплатный SSL-сертификат и публичный домен.